

```
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAXPAROLA 30
#define MAXRIGA 80
```

```
int main(int argc, char *argv[])
{
    int freq[MAXPAROLA]; /* vettore di contatori
delle frequenze delle lunghezze delle parole */
    char riga[MAXRIGA];
    int i, inizio, lunghezza;
    FILE *f;
```

```
for(i=0; i<MAXPAROLA; i++)
    freq[i]=0;
```

```
if(argc != 2)
```

```
{
    fprintf(stderr, "ERRORE: serve un parametro con il nome del file\n");
    exit(1);
}
```

```
f = fopen(argv[1], "r");
if(f==NULL)
```

```
{
    fprintf(stderr, "ERRORE: impossibile aprire il file %s\n", argv[1]);
    exit(1);
}
```

```
while( fgets( riga, MAXRIGA, f ) != NULL )
```



Inter-Process Communication

Shared Memory

Stefano Quer

Dipartimento di Automatica e Informatica

Politecnico di Torino

License Information

This work is licensed under the license



Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

① **BY:** Credit must be given to you, the creator.

② **NC:** Only noncommercial use of your work is permitted.

Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

③ **ND:** No derivatives or adaptations of your work are permitted.

To view a copy of the license, visit:

<https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1>

Introduction

- ❖ Shared memory allows two or more processes to share a given region of the main memory space
 - All processes can access the shared memory
 - Changes made by one process can be viewed by all other processes
- ❖ Shared memory is similar to file mapping
 - Shared memory is less restrictive but harder to use
 - Function **mmap** is a “recent” POSIX system call
 - Function **shmget** has the widest support and is the old System V model

Introduction

- Shared memory and file mapping are **faster** than pipes or messages queues
 - Data does not need to be copied between the client and the server and does not have to pass through the kernel
- Shared memory requires synchronization accesses to a given region among multiple processes
 - If the server is placing data into a shared memory region, the client should not try to access the data until the server is done
 - Semaphores must be used to **synchronize** shared memory access

Logic flow

❖ Logic flow to use the shared memory

- Generate an IPC key we can use the function **ftok**



- Get an identifier with **shmget**
- Attach the user to the shared memory with **shmat**
- Use pointers to manipulate the shared memory
- Detach the process from the memory with **shmdt**
- Remove the identifier, and possibly, control the shared memory with **shmctl**



Operations

Return value
Shared memory id, on success
The value -1, on error

System call	Meaning
<pre>int shmget (key_t key, size_t size, int flag);</pre>	Obtains a shared memory identifier given the key of the IPC object. The parameter size is the size of the shared memory segment in bytes. The parameter flag set the mode field of the IPC structure. See the example for further details.
<pre>int shmctl (int shmid, int cmd, struct shmid_ds *buf);</pre> Return value The value 0, on success The value -1, on error	Performs various operations on a shared memory. The memory is specified by its identifier (shmid). Parameter cmd specifies the command to be performed on the segment. As with function msgctl, it is possible to specify: PC_STAT, IPC_SET, IPC_RMID, or, when the process runs in super-user mode SHM_LOCK and SHM_UNLOCK.

Operations

System call	Meaning
<pre>void *shmat (int shmid, const void *addr, int flag);</pre> Return value Pointer to the memory, on success The value -1, on error	Attaches a process to its address space. The address in the calling process at which the segment is attached depends on: The addr argument specifies how to attach the segment (zero, to the first available address selected by the kernel; nonzero to the segment specified by addr or by $(\text{addr} - (\text{addr} \bmod \text{SHMLBA}))$)
<pre>int shmdt (const void *addr);</pre> Return value The value 0, on success The value -1, on error	When we are done with a shared memory segment, we call shmdt to detach it. Note that this does not remove the identifier and its associated data structure from the system. The identifier must be removed by calling shmctl with a command of IPC_RMID

Example

- ❖ Write two processes sharing 1KByte of memory and making some modification on it

Solution

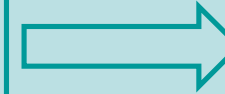
1 Reader + 1 Writer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```
#define SHM_SIZE 1024
```

```
int main (int argc, char *argv[]) {
    key_t key;
    int shmid;
    char *data;
```

(W) P₁



(R) P₂

The same process works as

- a reader (no parameter)
- a writer (writes the parameter on the shared memory)

Create a shared
memory segment of 1K

Solution

Make the
key

Here the file must exist

Create the segment

```
if ((key = ftok ("hello.txt", 5)) == -1) {  
    perror ("ftok");  
    exit (1);  
}  
if ((shmid = shmget (key, SHM_SIZE,  
                    0644 | IPC_CREAT)) == -1) {  
    perror ("shmget");  
    exit (1);  
}  
data = shmat (shmid, NULL, 0);  
if (data == (char *) (-1)) {  
    perror ("shmat");  
    exit (1);  
}
```

Attach the segment to
the local pointer **data**

Solution

Writer

Modify the segment, based on
the command line

```
if (argc == 2) {
    printf ("Writing to segment: \"%s\"\n", argv[1]);
    strncpy (data, argv[1], SHM_SIZE);
}
else
{
    printf("segment contains: \"%s\"\n", data);
}
if (shmdt(data) == -1) {
    perror ("shmdt");
    exit (1);
}
return 0;
}
```

Reader
Read the segment

Detach from the segment